

Agglomerative Hierarchical Clustering — Short Notes

Agglomerative Hierarchical Clustering is an **unsupervised ML algorithm** that builds clusters by **merging smaller clusters step-by-step**.

1. How it works

Initially, **every data point is its own cluster**.

A B C D E

↓

Merge closest clusters

↓

AB C DE

↓

Merge again

↓

ABC DE

↓

Merge

↓

ABCDE

So, it works **bottom → up**.

2. Dendrogram

A **dendrogram** is a tree diagram showing **which clusters were merged and at what distance**.

You can choose the number of clusters by **cutting the dendrogram at a particular height**.

3. Linkage Methods

These decide the distance between two clusters:

- **Single Link:** distance between the **closest** points.
- **Complete Link:** distance between the **farthest** points.
- **Average Link:** average distance between all point pairs.
- **Ward Link:** merges clusters while causing the **smallest increase in within-cluster variance**.

4. Important Parameters

AgglomerativeClustering(
 n_clusters=3,

```
linkage='ward'
```

```
)
```

- `n_clusters` → number of final clusters
- `linkage` → how clusters are merged
- `metric` → distance measure (API name in modern sklearn)

5. K-Means vs Hierarchical

K-Means

Need to choose K beforehand

Uses centroids

Fast for large data

Works best with roughly spherical clusters

No hierarchy

Hierarchical

Can choose clusters from dendrogram

Uses cluster distances

More computationally expensive

More flexible cluster structure

Produces dendrogram

Easy Memory

Agglomerative = Bottom → Up → Merge → Dendrogram → Cut → Clusters

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is an **unsupervised clustering algorithm** that groups points based on **density**.

Why DBSCAN?

Problems with K-Means:

- No need to specify **K (number of clusters)**.
- Detects **outliers/noise** automatically.
- Works with **irregular shapes** (circles, curves, etc.).

Two Important Parameters

1. Epsilon (ϵ):

- Radius around a point.

2. MinPoints:

- Minimum neighbors required inside ϵ .

ϵ = Radius

MinPoints = Minimum nearby points

Types of Points

1. Core Point

Has at least MinPoints inside ϵ .

- • •
- C •
- •

2. Border Point

Has fewer neighbors but lies near a Core point.

Core ---- Border

3. Noise Point

Far away from all clusters.

Cluster Noise

- • • • • x
- • • • •

DBSCAN Working

Start from a Core Point



Find nearby points



Expand cluster



Add density-connected points



Mark remaining points as Noise

Example

Suppose:

$$\epsilon = 2$$

MinPoints = 5

If a point has:

- 5 or more neighbors → Core
- Less than 5 but near Core → Border
- Isolated → Noise

Advantages ✓

- No need to choose K
- Detects outliers
- Finds arbitrary-shaped clusters
- Works well on noisy data

Disadvantages ❌

- Sensitive to ϵ and MinPoints
- Struggles with different cluster densities
- Slow for very large datasets

K-Means vs DBSCAN

K-Means	DBSCAN
Need K	No K needed
Uses Centroids	Uses Density
Poor with Outliers	Detects Noise
Circular clusters	Any shape

Easy Memory 🧠

DBSCAN = Density + Core + Border + Noise + No K